Operator Learning for Parametric Darcy Flow

Fourier Neural Operator (FNO) vs. MLP Baseline: Optimization, Scaling & Solver Comparison

Team PINNacles

Aryamann Srivastava, Varun Sathaye, Atishay Jain, Vedant S. Tiwari

Department of Aerospace Engineering, IIT Kanpur | November 2026

Objectives and Project Phases

What we set out to do, and what each stage delivered

Stage 1

Problem and plan

- Darcy flow as an operator-learning task
- Real PDEBench data, FNO vs MLP
- Evaluation plan and metrics

Stage 2

First real results

- Trained FNO and MLP on real data
- FNO error 0.052, MLP 0.08
- FNO about 1.6x more accurate

Stage 3

Make it better, and check it

- Better training, tuning, data and grid study
- Fair comparison with a classical solver
- Physics checks on the predictions

Objectives

1. Compare FNO and MLP fairly
2. Find what really drives accuracy
3. Test against a classical solver and the physics

Question for Stage 3: is the remaining error about the model, or about how we trained it?

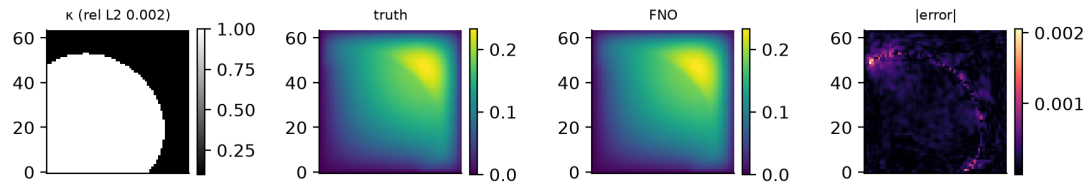
The Problem: Darcy Flow

Predict how pressure spreads through a porous material

Darcy's equation

$$-\nabla \cdot (\kappa \nabla u) = f$$

- Input: permeability map κ (two values, 0.1 and 1.0)
- Output: pressure field u
- Source: constant, f = 1
- Boundary condition: u = 0 on all four edges
- Goal: learn the map κ → u from 900 examples



Permeability, true pressure, FNO prediction, error

How physics is involved

- The PDE generated every training sample
- Its symmetry (flips, rotations) gives free extra data
- We check the predictions against the equation afterwards

A trained model answers a new permeability map almost instantly

Data and Training Setup

What the models see and how they are trained

10,000
real PDEBench simulations, 128 x 128

900 / 100 / 200
train / validation / test, at 64 x 64

5 seeds
for the final models, split fixed (seed 42)

The data

- Input: permeability map plus x and y coordinates
- Output: pressure field, same 64 x 64 grid
- Boundary condition is fixed by the PDE: u = 0 at the edges
- Inputs and outputs standardised with training-set statistics
- Error is always reported in physical units

The training

- Loss: relative L2 error per sample (slide 6)
- Optimiser AdamW, learning rate 1e-3, weight decay 1e-4
- Batch size 16, one smooth cosine learning-rate decay
- Flip and rotate augmentation (symmetries of the PDE)
- Studies: 5,700 to 20,000 steps; final models: 60,000

Same data, same loss and same budget for both models: only the network differs

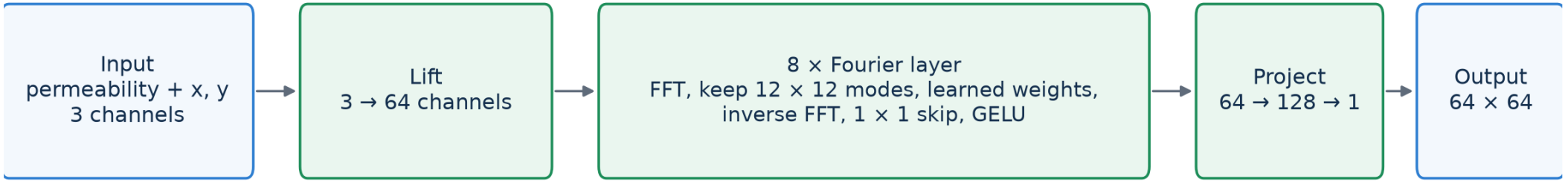
The Two Models

Same inputs and outputs, different way of looking at space

MLP baseline (100.7 M parameters)



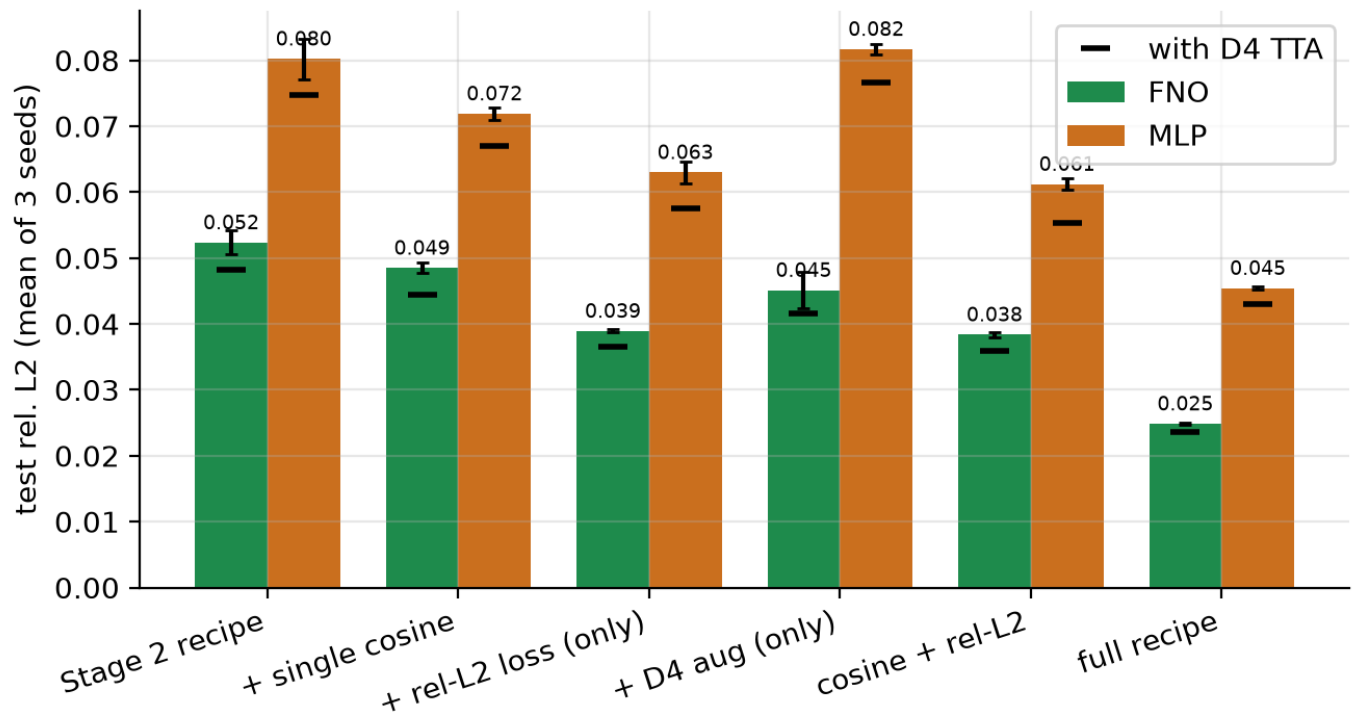
FNO (9.5 M parameters)



The MLP treats the map as a list of numbers; the FNO shares one operator over the whole grid

Finding 1: The Training Recipe

Changing how we train, not the model, cut the error roughly in half



1 Match the loss to the metric

Train on relative error, the number we report

2 One smooth learning-rate decay

Instead of repeated restarts

3 Flip and rotate the inputs

Free extra training data (symmetries)

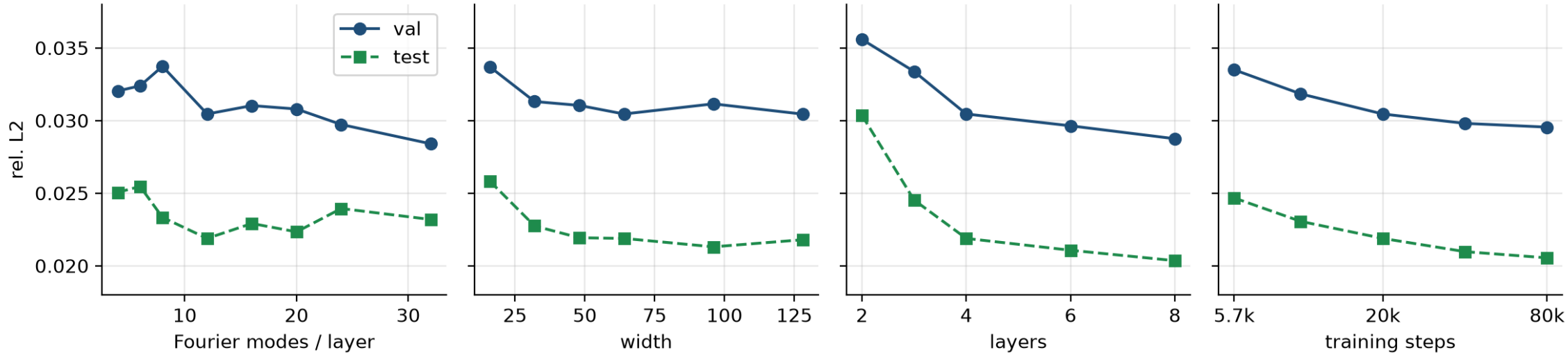
Loss function: relative L2 error per sample

$$\mathcal{L} = \frac{1}{B}\sum_{i=1}^{B}\frac{\|\hat{u}_i - u_i\|_2}{\|u_i\|_2}$$

Same model, same budget: FNO error 0.052 to 0.025, MLP 0.080 to 0.045

Finding 2: Architecture Matters Little

Width, depth and Fourier modes were tuned one at a time on validation data



Modes, width

almost no effect past modest values

Depth, training length

small, steady gains

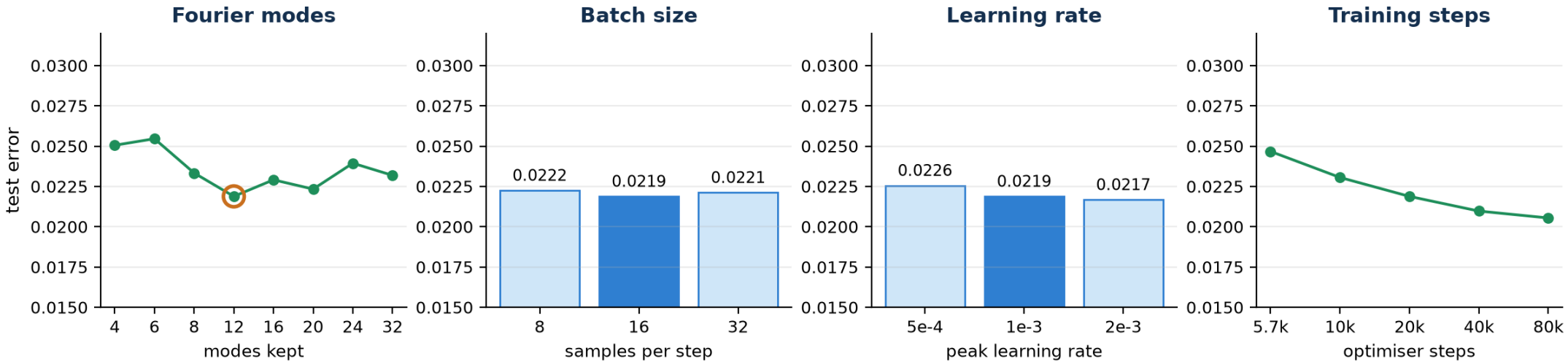
Final choice

9.5 M parameters, 8 layers

The training recipe was the big lever; architecture tuning added only a few percent

Why These Hyperparameters?

Each setting tested one at a time (one seed each, noise about 0.001); most barely matter



12 modes

Error is flat from 8 to 32 modes. 32 modes cost 7x the parameters and run 3.6x slower, for no gain.

Batch size 16

8, 16 and 32 give 0.0222, 0.0219, 0.0221: no real change. Kept from Stage 2.

Learning rate 1e-3

0.0005 to 0.002 stay within 0.001 of each other. A standard, safe middle value.

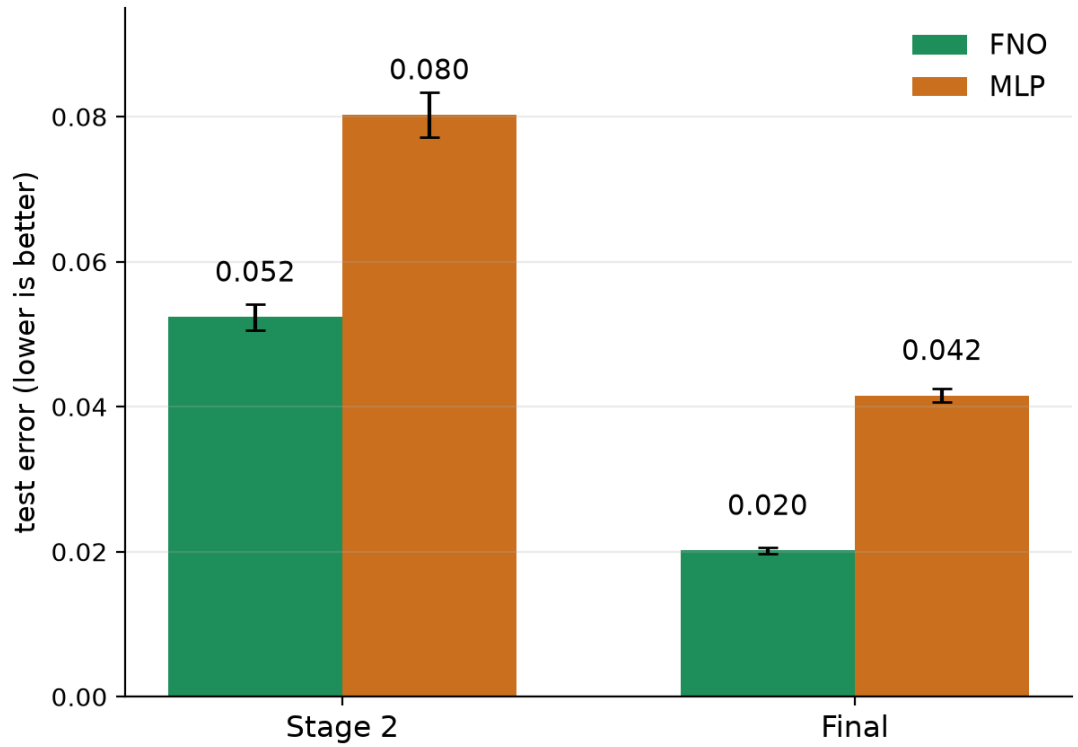
60,000 steps

More steps help slowly (0.0247 at 5.7k, 0.0205 at 80k). 60k fits our compute budget.

Flat curves mean a robust result: it does not hinge on a lucky setting

The Final Models

Both models tuned fairly, validation data only, 5 random seeds each



0.0202

FNO error, 9.5 M parameters

0.0415

MLP error, 100.7 M parameters

2x

more accurate

10x

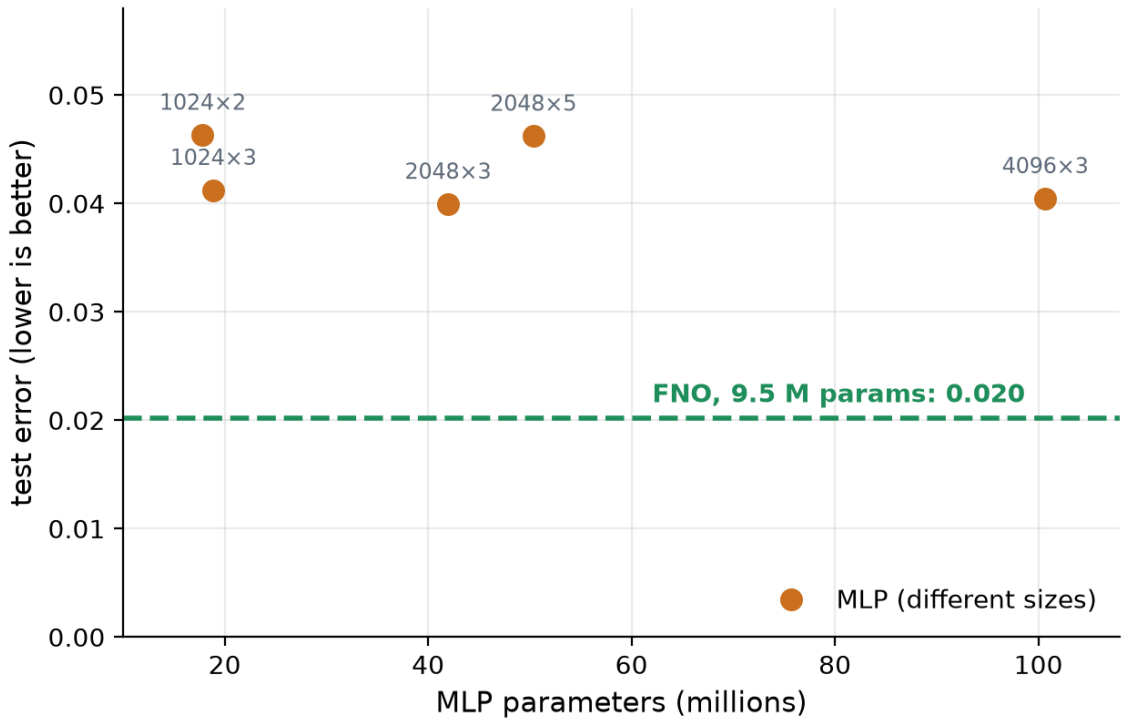
fewer parameters

The gap between FNO and MLP grew from 1.6x to 2x once both were trained well.

Spatial structure helps: the FNO wins with a tenth of the parameters

Why the MLP Cannot Do Better

It levels off at about twice the FNO error, whatever we change



1 **10x more parameters**

18 M to 100 M: error stays between 0.040 and 0.046

2 **More training**

20,000 to 60,000 steps: no gain (0.040 to 0.042)

3 **More data**

Helps up to about 5,000 samples, then flat near 0.035

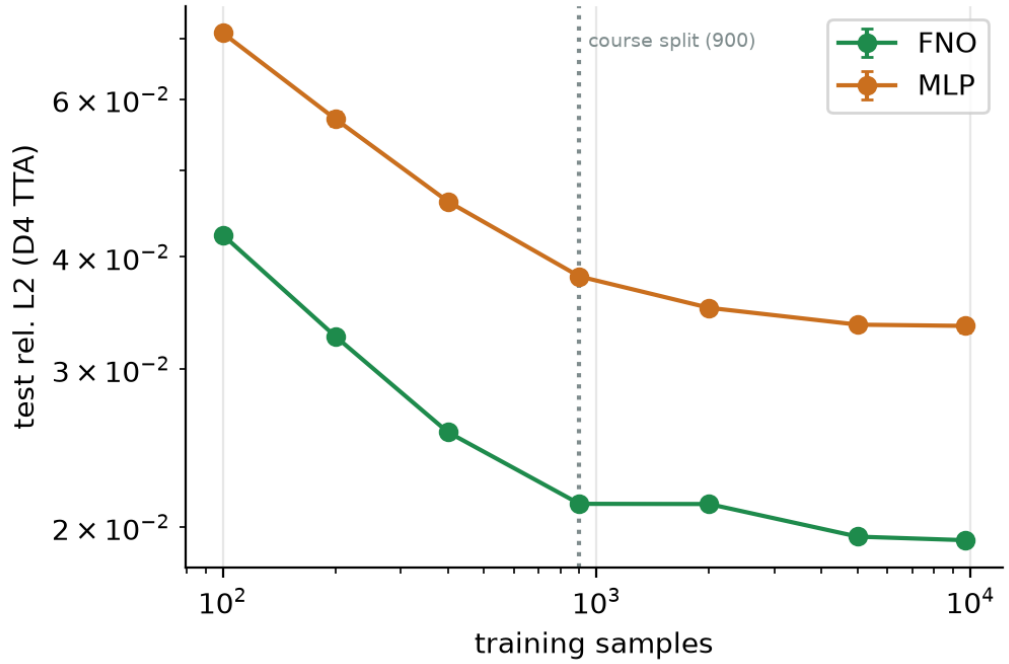
Likely reason (our interpretation)

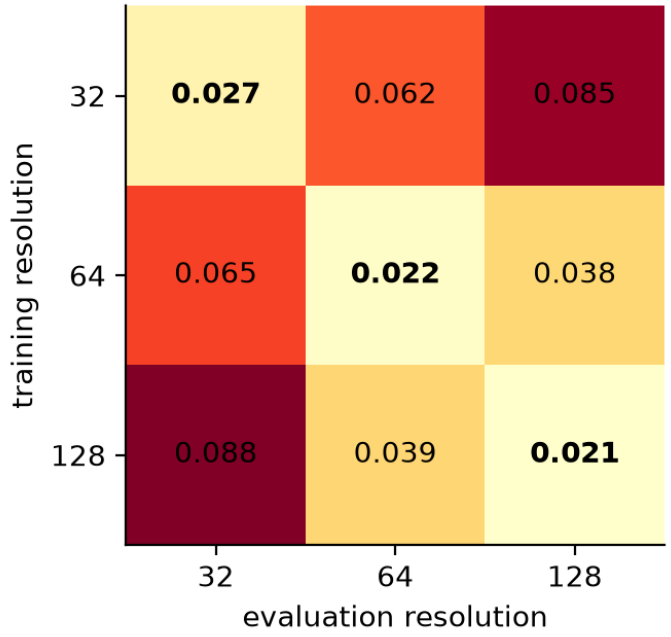
The MLP flattens the grid, so it has no notion of neighbours and must relearn the same local physics at every pixel. The FNO reuses one operator everywhere.

The FNO with only 400 training samples (0.026) already beats an MLP trained on 9,700 (0.036)

Finding 3: Data and Resolution

The FNO needs less data and can be used on other grid sizes, with a cost





How to read the grid

Rows: grid used for training

Columns: grid used for testing

Numbers: test error

400 samples

The FNO already beats an MLP trained on 9,700

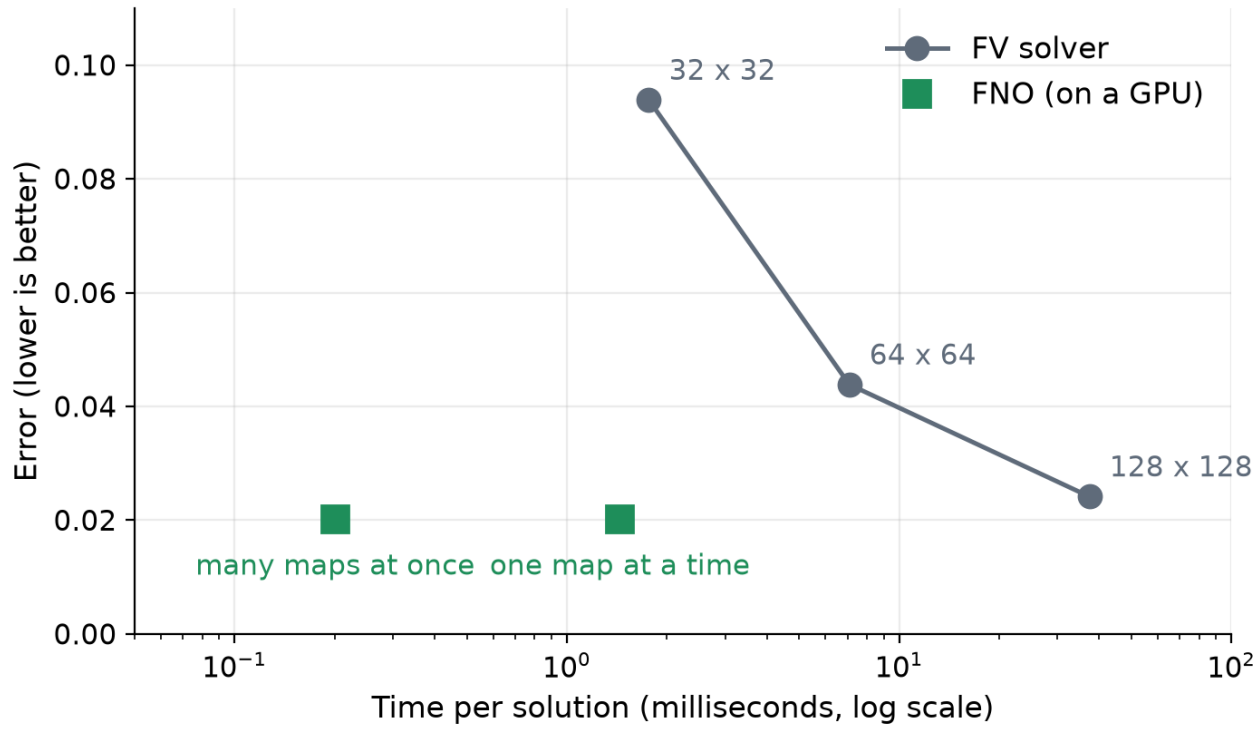
Other grid sizes

2 to 3x worse on a new grid: edges and boundary are sampled differently

More data helps until about 5,000 samples, then error levels off near 0.02

FNO versus a Classical Solver

Neural network versus the classical calculation



0.020 vs 0.044

error: FNO vs solver (64 x 64 grid)

5x to 35x

faster than the solver, on a GPU

0.2 ms

per solution, batched on a GPU

Timing note: on one CPU core the solver is faster (7 ms vs 37 ms); see report, Section 7.

Best used on GPUs and in batches, whenever many solutions are needed

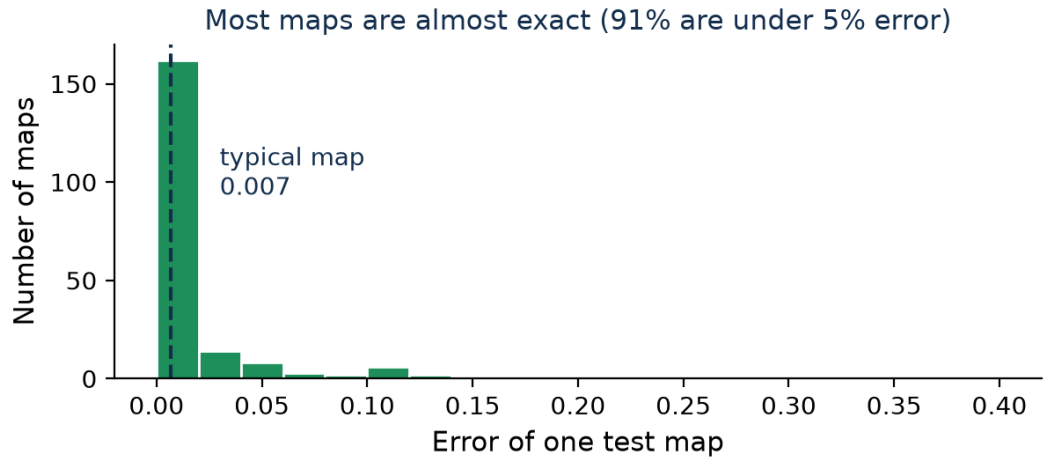
Does It Obey the Physics?

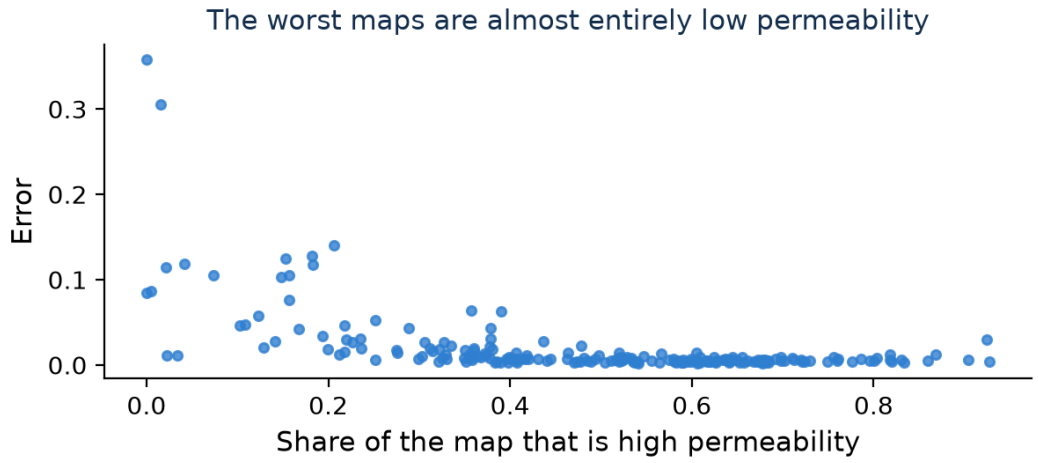
Do the predictions behave like real pressure?

0.1157
edge pressure level (real data 0.1159)

1.7%
error in average pressure

0.0015%
pixels with negative pressure

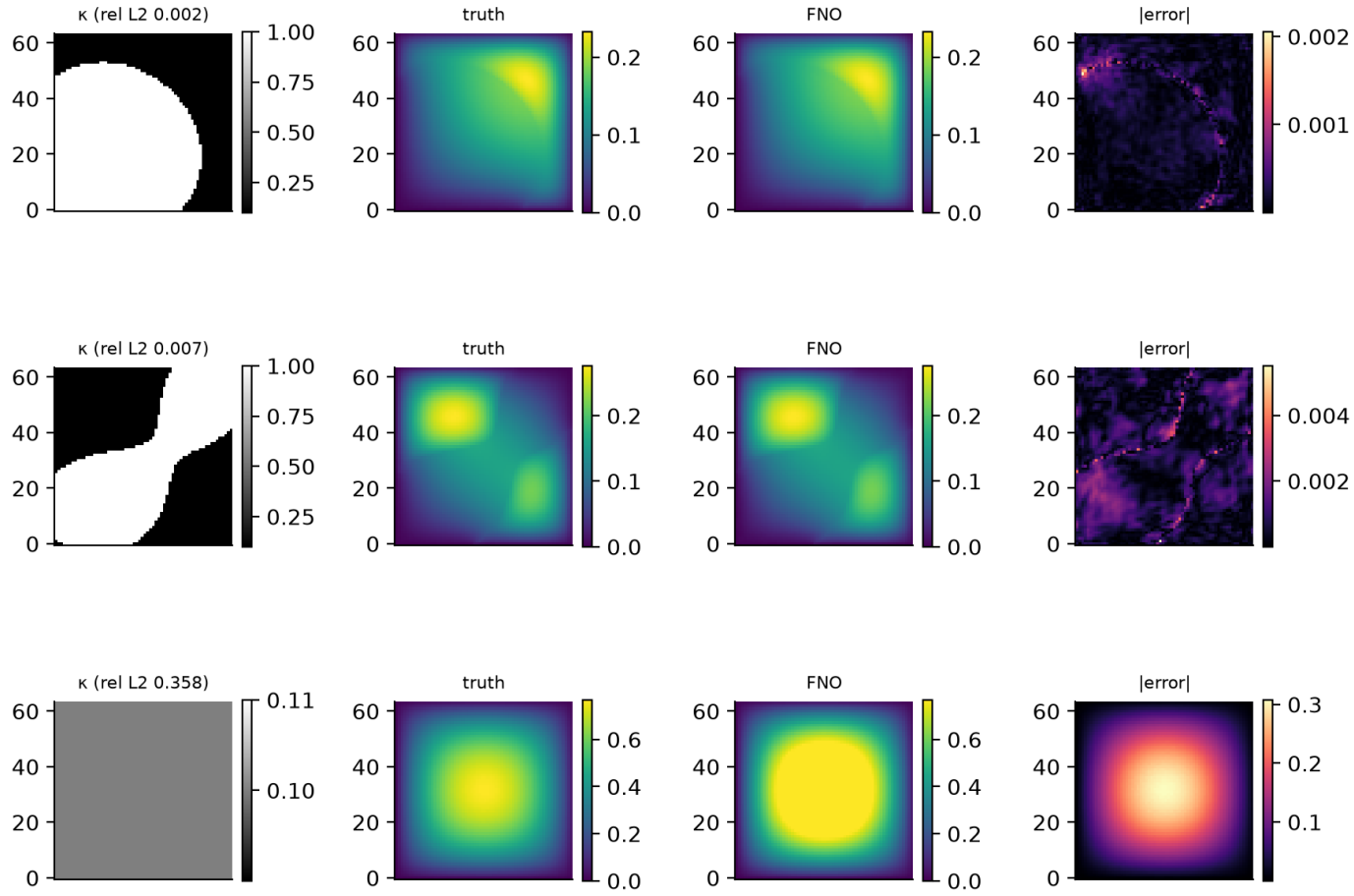




Worst cases: almost uniform maps, where the pressure height is hard to guess

Example Predictions

Best, typical and worst test cases of the final FNO



1 Best

Error 0.2%, almost no visible error

2 Typical

Error 0.7%, shapes match closely

3 Worst

Uniform permeability: right shape, wrong height (36%)

Summary and Limitations

What we learned, and what we do not claim

What we learned

1. **Training mattered most**
   Error halved with a better recipe
2. **FNO beats a tuned MLP**
   2x more accurate, 10x fewer parameters
3. **Strong against a solver**
   More accurate at the same grid size

Limitations

1. **One PDE, one dataset**
   Darcy with f = 1 and two permeability values; trends may not transfer
2. **Agreement with the data, not the exact PDE**
   The FNO learns this generator's quirks, e.g. its boundary offset
3. **Tied to the training grid**
   Other grid sizes are 2 to 3x worse; not resolution-free here
4. **Speed depends on hardware**
   The gain is on a GPU in batches; the Stage 2 speed-up claim was withdrawn
5. **Hard cases remain**
   Nearly uniform permeability: right shape, wrong height (36% error)

Better training beat a bigger model, and the FNO stayed accurate and fast

Team Contributions & AI Declaration

Individual ownership and adherence to AE646 academic integrity guidelines

Aryamann Srivastava

- Stage 3 pipeline & D4 augmentation
- Recipe & architecture studies; final models
- Finite-volume solver & latency benchmarks
- Reports, decks & code packaging

Varun Sathaye

- PDEBench train/val/test split verification
- Normalisation & Stage 2 baseline checks
- Solver calibration & timing review
- Section 7 results & presentation

Atishay Jain

- Training recipe & mode/ width/depth sweeps
- Ablation tables & Pareto analysis
- Model-selection protocol verification
- Section 4-5 results & presentation

Vedant S. Tiwari

- Data-scaling & cross-resolution transfer
- Physical diagnostics & residual checks
- Failure mode analysis (uniform kappa)
- Section 6 & 8 results & presentation

AI-Tool-Use Declaration

AI tools were used for coding assistance, debugging, experiment orchestration (Colab/Kaggle GPU runtimes), and LaTeX/PPT drafting. All numerical results come from real runs on real PDEBench data; no results or metrics were generated or invented. Every line of code, equation, and conclusion is fully owned, verified, and defendable by all team members in the viva.